Sistemas Distribuidos I



Entrega II: Tolerancia Fallas

PADRÓN	ALUMNO	MAIL
104414	Correa, Valentina Laura	vcorrea@fi.uba.ar
104046	Andrade, Agustin Ariel	aandrade@fi.uba.ar
101231	Dimartino, Pablo Salvador	pdimartino@fi.uba.ar

Grupo: 20
Cuatrimestre: 1c 2025
Fecha de entrega: 26 de Junio

Índice

Sistemas Distribuidos I 0
Entrega II: Tolerancia Fallas 0
Índice 1
Introducción 2
Detalles de la solución 2
- TCP 3
  - Inicio de AbstractAggregator 3
  - Handleo en Joiners 4
- Eliminación de datos de clientes desconectados 4
- Monitores 4
- Compresión de archivos 5
- Persistencia de mensajes 6
- Nueva interacción Joiner - Aggregator 6

Estructura del código 7
- Worker 7
- Abstact aggregator 7
- Filters 8

Scripts 9
- Comparación de resultados 9
- Corrida con parámetros automatizados 9
- Generación de archivos 9

Diagramas 10
- Vista Física 10
  - Diagrama de robustez 10
  - Diagrama de despliegue 10
- Vista de procesos 11
  - Diagrama de actividad 11
  - Diagrama de secuencia 14
- Vista de desarrollo 17
  - Diagrama de paquetes 17
- DAG 18
- Otros 19
  - Eleccion de lider 19

Introducción

El sistema distribuido desarrollado tiene como objetivo procesar datos de películas y valoraciones de usuarios a partir de tres archivos: movies_metadata.csv, ratings.csv y credits.csv. A partir de esta información, nuestro sistema responde cinco consultas específicas:

1. Películas y sus géneros de los años 00' con producción Argentina y Española.
2. Top 5 de países que más dinero han invertido en producciones sin colaborar con otros países.
3. Película de producción Argentina estrenada a partir del 2000, con mayor y con menor promedio de rating.
4. Top 10 de actores con mayor participación en películas de producción Argentina con fecha de estreno posterior al 2000
5. Average de la tasa ingreso/presupuesto de películas con overview de sentimiento positivo vs. sentimiento negativo, para películas de habla inglesa con producción americana, estrenadas a partir del año 2000

El sistema utiliza como unidad de trabajo central una cantidad de workers configurable según el tamaño del archivo a procesar, dividiéndolo en subtareas para el procesamiento distribuido paralelo en varias etapas.

Detalles de la solución

El sistema implementa una arquitectura distribuida orientada al procesamiento de flujos de datos mediante la colaboración entre nodos autónomos.

Se pueden distinguir cuatro etapas en el procesamiento: la división en subtareas, el procesamiento de dichas tareas, el agregado de los resultados y la creación de nuevas subtareas en los casos que corresponda.

La solución se apoya, por un lado, en un conjunto de *workers* no especializados que conforman las unidades de procesamiento principales. Estas resuelven tareas cuyo volumen crece proporcionalmente al tamaño del archivo de entrada. Por otro lado, hay unidades de agregado que toman las respuestas parciales de dichas tareas y las combinan en una respuesta total. Suponemos que estas operaciones de agregado tienen una complejidad relativamente mucho menor que las tareas principales.

Estas dos partes se comunican a través de un *middleware* que provee operaciones de envío, recepción y suscripción a grupos lógicos de trabajo, permitiendo desacoplar la lógica de negocio de la comunicación.

La arquitectura fue diseñada para favorecer la escalabilidad horizontal, el desacoplamiento de responsabilidades y la extensibilidad funcional a partir de nuevos tipos de nodos o consultas.

Para el manejo del fin de archivo, se trabajará con un envío de paquetes distribuidos de forma tal que incluyan los resultados necesarios, las líneas enviadas parcialmente y el total de líneas del archivo. El total de líneas será inicialmente 0, hasta que el cliente que envía el archivo lo calcule y lo incluya en el último paquete. Luego, cada agregador irá sumando las líneas parciales que reciba y, en el momento en que conozca el total de líneas, lo utilizará para completar el procesamiento.

TCP

Se introdujo una nueva comunicación entre los joiners y los aggregators, en la cual interactúa el protocolo de comunicación TCP.

Algunos desafíos encontrados durante esta nueva funcionalidad fueron el hacer tolerante a fallas esta comunicación, en particular ante la caída del aggregator. Se resolvió hacer que la consulta espere una respuesta sincrónicamente y sea resuelta en el momento en el que vuelva a levantar el nodo caído.

Inicio de AbstractAggregator

Cuando el abstract aggregator (implementado por los workers) se inicia, se ejecuta la funcion de load_processed_batches(). La lógica principal de esta consiste en levantar el archivo en donde se encuentran los registros previamente persistidos y comenzar a guardar las transacciones que fueron realizadas en su totalidad, es decir que para ese batch_id existe tanto un log de BEGIN_TRANSACTION como de END_TRANSACTION.

Esto se realiza hasta la anteúltima transacción. Cuando se llega a la última transacción, se evalúa si esta alcanzó a completarse o no (resolve_unfinished_transaction). Aquí se realiza una consulta desde el joiner al aggregator para saber si la tiene que procesar (no fue procesada por ningún otro joiner) o no. Esto lo resuelve enviando un mensaje por TCP con el type batch_processed, en donde el aggregator responderá con la instancia de joiner_id ejecutor asociada si es que existe. En base a esta respuesta se decide si se debe procesar esta última transacción o no.

Se puede ver este escenario de manera gráfica en un diagrama de secuencia sobre el final de este informe.

Handleo en Joiners

Asi mismo, esta misma lógica también ocurre en los joiners al momento de recibir un mensaje de credits (handle_message). Al recibirlo, se realiza esta misma consulta al aggregator para validar que aún no haya sido procesado por otro nodo joiner. El joiner se queda esperando esta respuesta y en base a esto decide si enviar un ack para ese batch id de mensaje a rabbit (ya fue procesado) o bien realizar el procesamiento del mismo.

Eliminación de datos de clientes desconectados

Para asegurarnos que la eliminación de datos de un cliente desconectado se complete utilizamos la estrategia de “mensajes envenenados”. Al obtener un error cualquiera en el handler de conexión de cliente, enviamos un mensaje con el acumulado actual enviado, fingiendo una petición correcta y luego descartando los resultados obtenidos.actividad

Monitores

Uno de los principales desafíos en esta etapa del sistema es establecer un mecanismo confiable para la recuperación de servicios caídos, sin que interferir con otros componentes en desarrollo. Para esto realizamos:

Heartbeats

Cada servicio se implementa como una instancia de una clase base Worker, la cual incorpora un componente encargado de enviar heartbeats periódicos a un conjunto de monitores. Estos heartbeats son mensajes UDP que actúan como señales de vida los cuales permiten a los monitores determinar si un servicio está activo.

Registro y Descubrimiento de Servicios

Para que los monitores sepan qué servicios deben supervisar, se estableció un mecanismo de registro de heartbeats por servicio. El problema es que esta estrategia enfrenta una limitación: si el monitor líder falla, se pierde el estado de qué servicios estaban registrados. Para cubrir este escenario (y también para detectar servicios que nunca lograron iniciar correctamente) se introdujo a través de variables de entorno generadas dinámicamente por docker-compose. Este enfoque asegura que los monitores siempre conozcan la totalidad de los servicios críticos del sistema, incluso en situaciones de arranque o recuperación.

Elección de Líder

Dado que solo un monitor debe encargarse de procesar heartbeats y gestionar reinicios, implementamos un mecanismo de elección de líder utilizando el algoritmo Bully. Ante una desconexión del líder actual, los monitores restantes inician el proceso de elección para designar uno nuevo de forma automática y coordinada.

Un desafío adicional es la comunicación entre servicios y el nuevo líder tras una elección, ya que puede no existir una conexión directa. Para resolver esto, se habilitó una lógica en los senders que les permite consultar a cualquier monitor (elegido aleatoriamente y de forma periódica) cuál es el líder actual.

Cooldown

Se realiza un cooldown sobre los servicios recientemente levantados para darle tiempo a que realice su inicio y pueda enviar su señal de heartbeat.

Compresión de archivos

Para reducir el volumen de datos en disco se realiza una compresión periódica del archivo de resultados persistidos en disco. Aprovechándose de que ya tenemos en memoria un agregado de los resultados actuales, reemplazamos el archivo actual por uno equivalente el cual contiene la información necesaria. Esto se realiza en el método *compact_log_for_client* de la clase *abstractaggregator.py*.

Para realizar esto, aprovechando que ya está abierto el *file descriptor* del archivo de persistencia de resultados obtenemos su tamaño evitando una doble lectura a disco. Si el archivo supera un tamaño establecido (Hoy en 100kB) se realiza la compresión.

El proceso de persistencia comienza creando un archivo con sufijo *_compacted*, luego creando una entrada ID;some-batch-id; para cada batch ya persistido del cliente y luego creando una entrada con la misma estructura de un resultado al final del archivo.

Finalmente realiza un reemplazo seguro borrando el archivo actual y renombrando el *_compacted* al nombre original.

Si se requiere recuperar los mensajes, se valida primero que no exista un compactado baldío. De ser así toma este como válido, sino continúa utilizando el anterior. Un ejemplo de un log con 3 IDs comprimido tiene la forma

ID;1-XGSJ-rating-5280000-N-YIXC

ID;1-XGSJ-rating-90000-N-INPD

BEGIN_TRANSACTION;1-XGSJ-rating-20000-N-QUJT;{"client_id": "1-XGSJ", "result": {"45722": {"rating_sum": 23.5, "votes": 7}, "1653": {"rating_sum": 27.5, "votes": 7}, "1956": {"rating_sum": 7.0, "votes": 2}, "16": {"rating_sum": 29.0, "votes": 7}}, "batch_size": 10000}

END_TRANSACTION;1-XGSJ-rating-20000-N-QUJT

Persistencia de mensajes

Aprovechando la premisa de que no detenemos el contenedor de *RabbitMQ* utilizamos la estrategia de persistir lo antes posible los mensajes y después realizar el ack a el middleware. De esta manera se intenta minimizar la cantidad de doble lectura de mensaje. Un agregador intenta tomar “propiedad” de un mensaje lo antes posible.

Pero esto nos expone a mensajes duplicados: Si entre la persistencia de un mensaje y la emisión del *ack* el contenedor muere otro joiner podría obtener dicho mensaje. Para evitar esto en los agregadores de las consultas 1, 2 y 5 se realiza la validación de mensajes duplicados. Los joiners necesitan una validación centralizada, visto en la sección de TCP.

Para persistir un mensaje se crea un archivo con prefijo el nombre del cliente y sufijo un string distinto para cada caso. Una entrada de resultado puede ser, como ejemplo:

BEGIN_TRANSACTION;1-XGSJ-rating-20000-N-QUJT;{"client_id": "1-XGSJ", "result": {"45722": {"rating_sum": 23.5, "votes": 7}, "1653": {"rating_sum": 27.5, "votes": 7}, "1956": {"rating_sum": 7.0, "votes": 2}, "16": {"rating_sum": 29.0, "votes": 7}}, "batch_size": 10000}

END_TRANSACTION;1-XGSJ-rating-20000-N-QUJT

Para recuperar archivos primero se lee el directorio raíz, se consulta por todos los archivos con el sufijo pedido y del mismo nombre se obtiene el ID de cliente. Luego se itera buscando transacciones completadas, evitando cualquiera que no sea válida. Cada transacción es procesada por el método de agregación de cada clase.

La última transacción se procesa por un método aparte para poder realizar implementaciones distintas en los distintos aggregators, evitando agregarla si la consulta por tcp responde que ya fue consumida por otro worker.

Nueva interacción Joiner - Aggregator

En esta nueva entrega se tuvo que hacer un rediseño de la lógica actual que involucra la relación entre los nodos joiners (ratings y credits) y sus respectivos aggregators. En este

caso se resolvió a una arquitectura que permite múltiples nodos joiners, los cuales incorporan la tolerancia a fallas de la manera en la que se describe en la sección de tcp, y un aggregator que irá contabilizando la totalidad de resultados de todos sus joiners conectados que existan en ese momento.

Cada uno de los joiners irá enviando sus resultados propios, los cuales al no envolver la totalidad del archivo serán parciales. Así mismo también se irán enviando cantidad de procesados y totales. Es de esta manera entonces como el aggregator puede comprender los de todos los joiners y reconocer en qué momento finalizó el procesamiento.

Repasando esto último, la forma en que procesamos nuestros archivos incluye un mecanismo de control adicional junto con la información de películas en cada batch. Específicamente, cada mensaje enviado por los workers contiene, además de los datos del batch, un campo adicional que indica la cantidad de líneas procesadas por ese worker. En el último mensaje correspondiente a un cliente, este campo también incluye el total acumulado de líneas esperadas.

Este esquema permite que el aggregator tenga visibilidad del progreso y pueda determinar con precisión cuándo se han recibido todos los datos necesarios. Una vez alcanzado ese punto, el aggregator puede consolidar la información y enviar el resultado final al client_decodifier, que representa al cliente interno del sistema.

Estructura del código

Worker

Un worker es una estructura genérica la cual es implementada tanto por el client_decodifier como así también por el abstract aggregator y los filtros. Este reúne funcionalidades compartidas tales como comunicación con rabbitmq. Además también tiene dos métodos abstractos (start y close) para que sea implementado por cada una.

Abstact aggregator

Como mencionamos anteriormente, el abstract aggregator también hereda de Worker. Este es implementado por los aggregators particulares de cada consulta, debido a que reúne lógica compartida por todos:

- Carga de mensajes guardados luego de una recuperación

- Handleo de mensajes con lógica y contabilización de batches procesados y faltantes, para descular si se está en el fin del procesamiento o no
- Limpieza de recursos utilizados
- Persistencia de datos
- Validación de backups logs
- Envío de mensajes de resultado
- Resolución de transacciones incompletas
- Compactación de log para el cliente

La decisión de realizar esto en un único lugar se hizo con la intención de unificar comportamiento y facilitar el debug.

Asi mismo, incluye funciones abstractas genericas las cuales implementadas por cada uno de los aggregators:

- process_message
- aggregate_message
- create_final_result
- create_consumer
- create_producer

Decidimos realizar esto último para ganar mayor visibilidad, y que sea mas facil la comprensión entre los distintos archivos.

Filters

Se utiliza otra clase abstracta para los filtros. Estos heredan de una clase padre *filter*, la cual también hereda de *worker*. Los filtros buscan resolver una única tarea, sin mantener estado:

- Filtrar películas argentinas
- Filtrar películas españolas
- Filtrar películas de los años 2000
- Filtrar películas que no tengan colaboración con otros países

Asimismo también se tiene el filtro de *sentiment analysis* que se encarga de calcular el promedio de la tasa ingreso/presupuesto de películas con overview de sentimiento positivo vs sentimiento negativo.

Adicionalmente se incorporo tambien un filtro generico para las movies (main_movie_filter), el cual limpia el dataset.

Scripts

Para la asistencia en el deploy y el testing del trabajo práctico se utilizan algunos scripts:

Comparación de resultados

Para la comparación de resultados primero se ejecuta el script *create_results.py* el cual genera con la misma estructura que los files.

Corrida con parámetros automatizados

Se tienen los archivos de configuración inicializados con config en su nombre que permiten configurar facilmente la cantidad de nodos que se quieren utilizar en la ejecucion, el nivel de log y si se lo quiere matar o no con el script que automatiza las caidas.

Generación de archivos

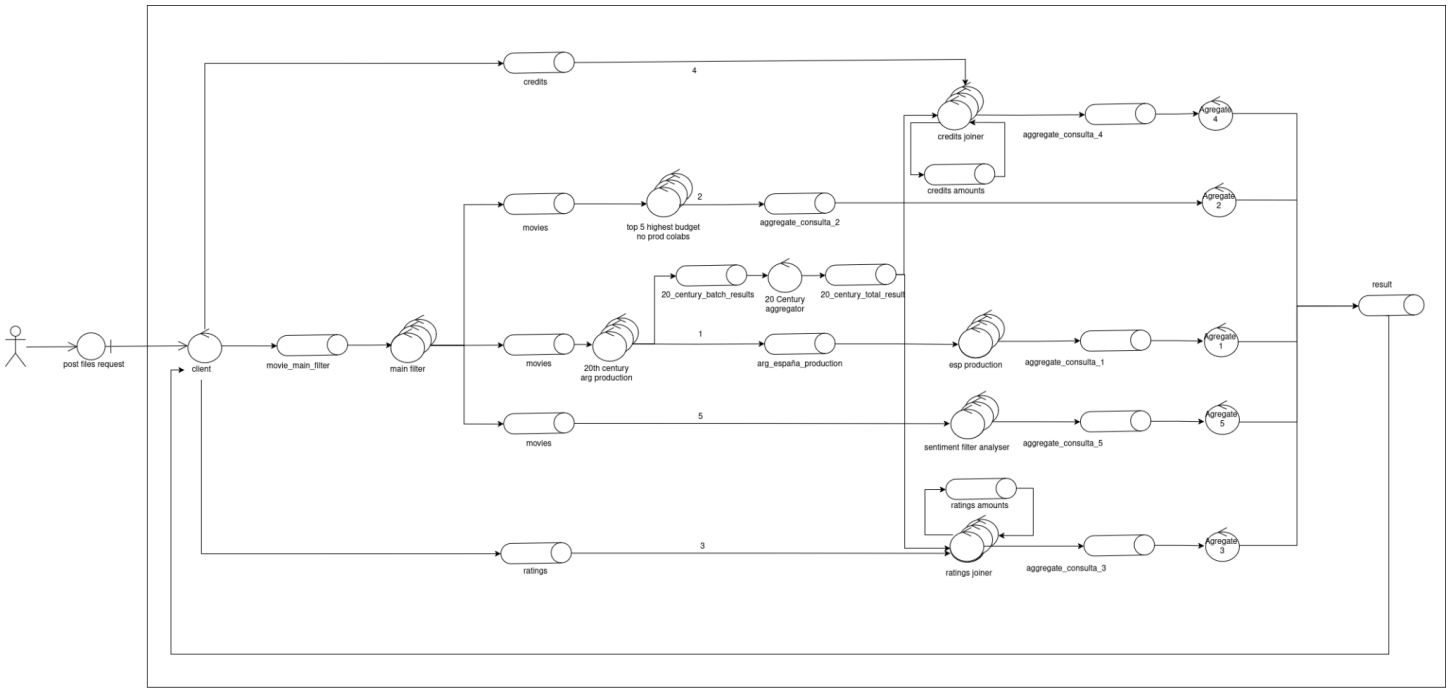
Para obtener un docker compose a medida de las necesidades que vayan apareciendo, fue necesario crear un script que a partir de un archivo de configuraciones genere un compose correspondiente

Diagramas

Vista Física

Diagrama de robustez

En el presente diagrama se puede observar como el controller representante del cliente separa los archivos en *chunks* y los envía a la cola correspondiente en base al tipo de archivo. Cada una de estas colas será encargada de enviar estos chunks al worker correspondiente, quien realizará el filtro correspondiente. En algunos casos, como por ejemplo en las consultas 3 y 4 vemos que a su vez estos derivaran a una nueva cola que desencolara en un joiner. Este será el encargado de consumir de ambos lugares y hacer el mapeo que se necesite. Todos los resultados desencadenarán en un nodo aggregator. Este, al ser el último del flujo será el encargado de enviar aquella respuesta al cliente.

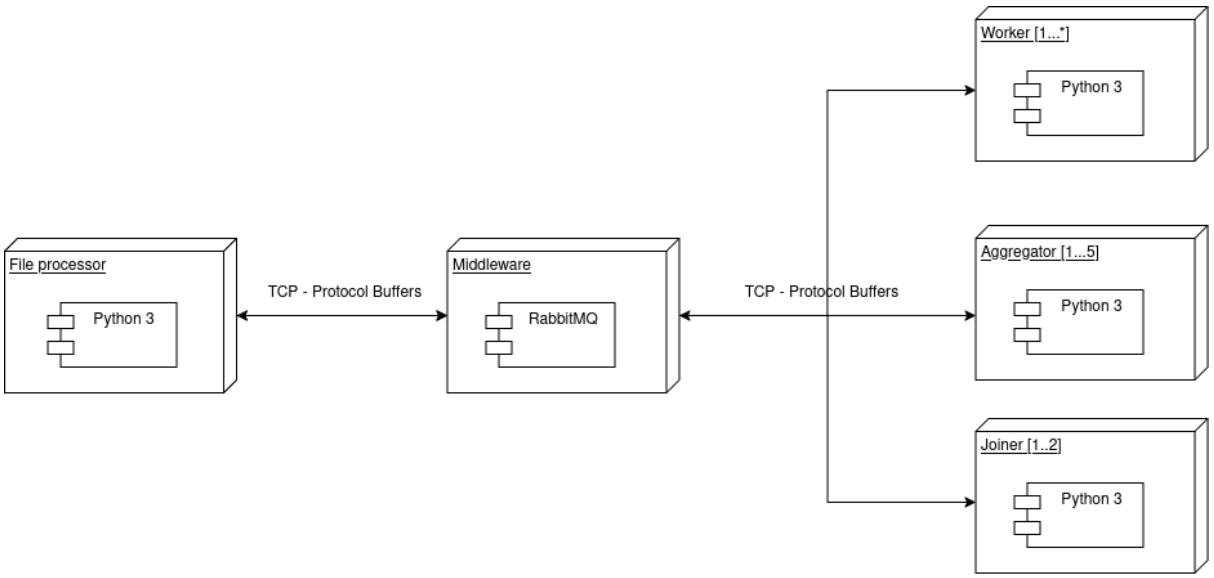


Actualmente nuestro sistema puede escalar 9 nodos de un total de 14, resultando en un ~64% de escalabilidad total de todo nuestro sistema distribuido.

Diagrama de despliegue

En nuestro diagrama de despliegue distinguimos 5 componentes principales. Por un lado el procesador de archivos el cual será el encargado de levantar cada uno y enviarlos a través del middleware. Este último a su vez será quien escuche y reciba mensajes de los workers, aggregators y joiners.

La comunicación del middleware será implementada mediante colas de prioridad RabbitMQ. El código de los demás componentes será implementado en Python.



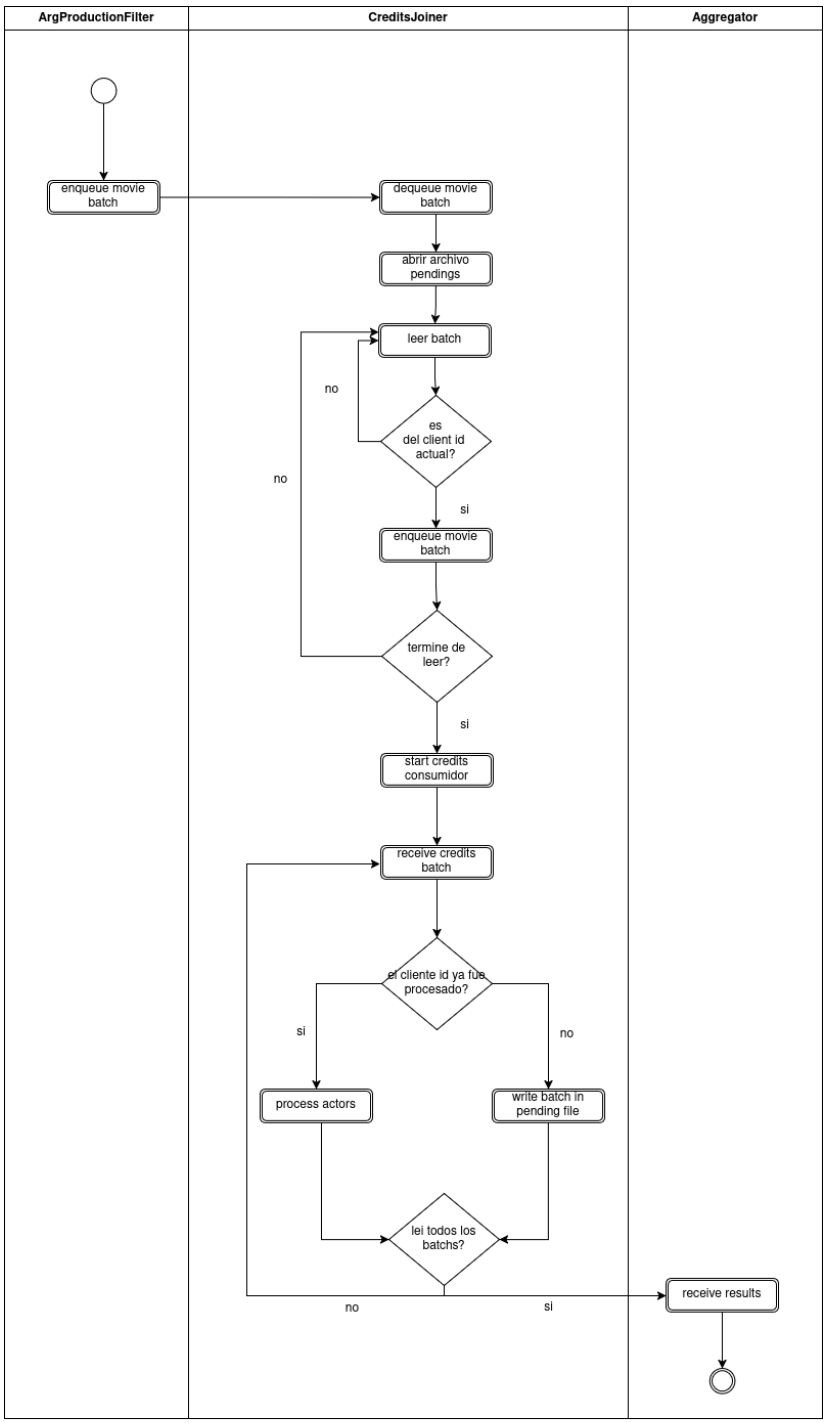
Vista de procesos

Diagrama de actividad

Las request comienzan a handlear desde el cliente. Como vimos el detalle en el diagrama de robustez este realiza las divisiones en n chunks y las irá encolando a los distintos filtros según corresponda. Estos pueden a su vez encolar en nuevos filtros, como es en el caso expuesto.

Podemos ver que mientras aún haya elementos disponibles por desencolar en la cola de tareas los workers continuarán haciéndolo. A medida que se vayan obteniendo resultados parciales estos son encolados en el aggregator. Una vez procesados y almacenados todos los chunks, se realiza la respuesta final hacia el cliente.

En el presente diagrama se muestra cómo se procesan los batches de clientes que aún no fueron anunciados.
Imaginemos el escenario en donde tenemos 2 clientes “A” y “B”. El cliente “A” llego primero, con lo cual luego de anunciarse esta habilitado a consumir batches de ratings. Al estos estar en una cola común para todos los clientes, es probable que no todo lo que “A” popee sea para él. De acuerdo con esto, lo que se hace en esta situación es: en el momento en donde se popea un batch de credits que no sea propio, este se escribe en un csv de registros pendientes -asociados a clientes que están por llegar-.
En el futuro, cuando se anuncie la llegada de “B”, lo que hará éste será levantar ese archivo e ir leyendo cada registro: si el batch es suyo, lo encolara nuevamente en la cola de credits, dado que ahora no habrá conflictos ya que pasa a estar habilitado para consumirlo. Caso contrario (hay más batches de otros clientes que aún no llegaron), se reescribe en un nuevo archivo. Este último será el nuevo archivo de pendings.



En el siguiente diagrama de actividad, se puede ver como resulta la interacción dentro del credits joiner cuando inicia y se propone recuperar las peliculas. Comienza indagando dentro de su directorio ya que habra uno por cada cliente conectado. Si este tiene un

formato de línea válido, es decir BEGIN_TRANSACTION o bien END_TRANSACTION, se recupera la informacion y se carga como se puede ver a continuacion.

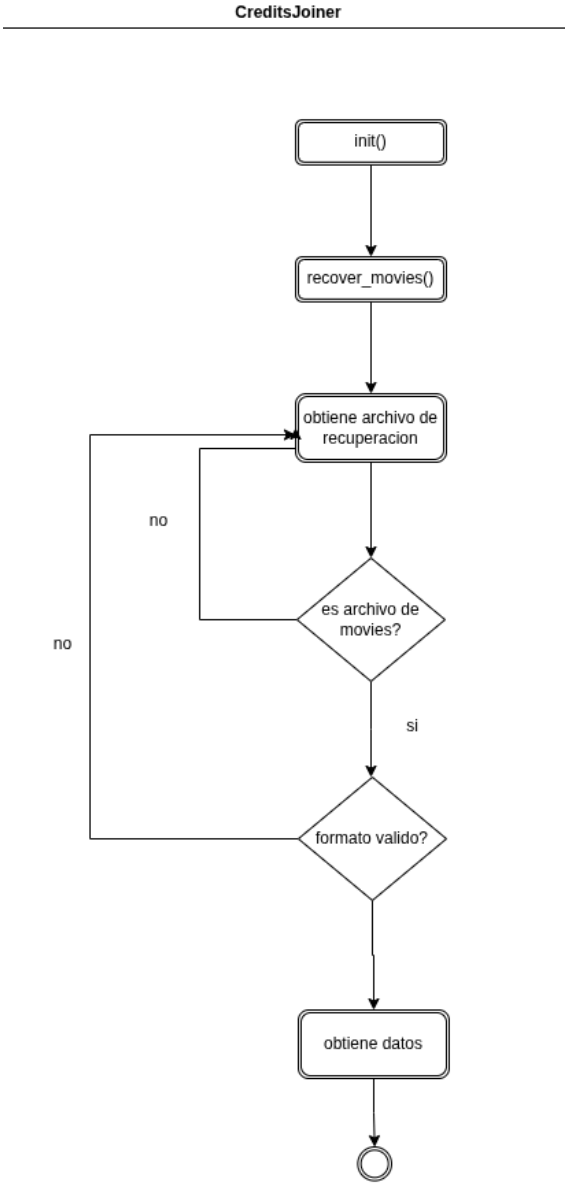
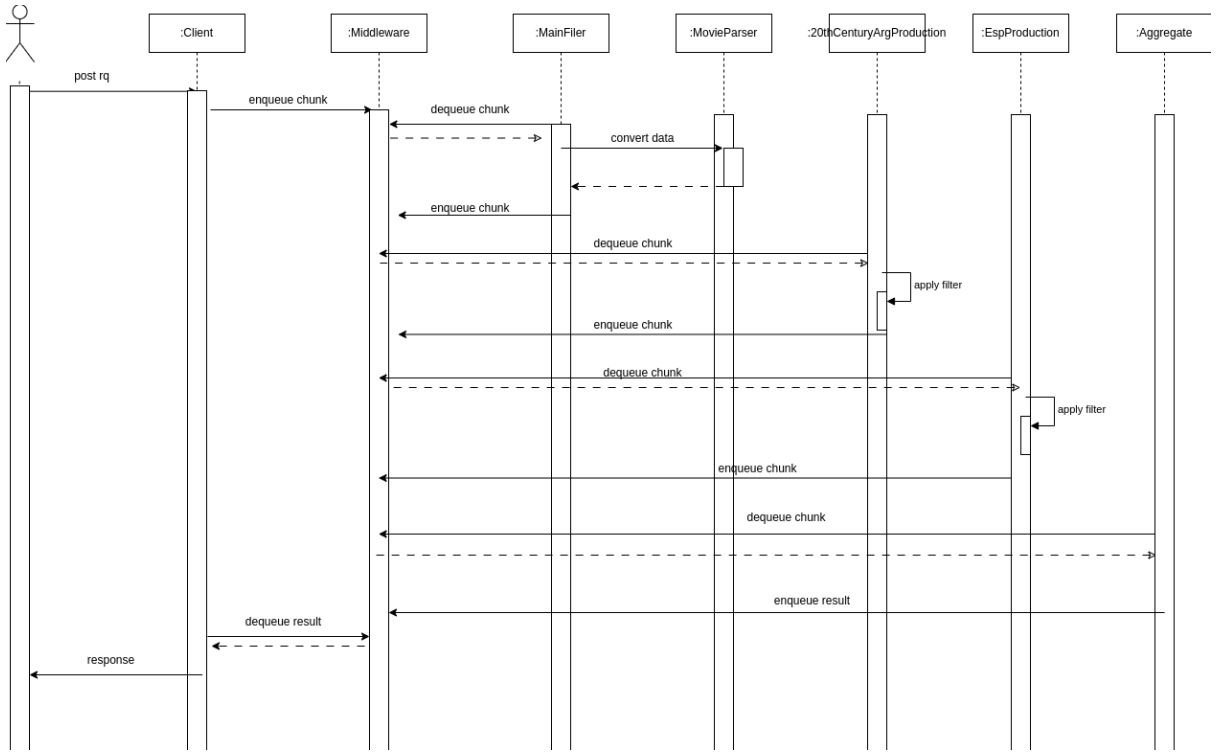


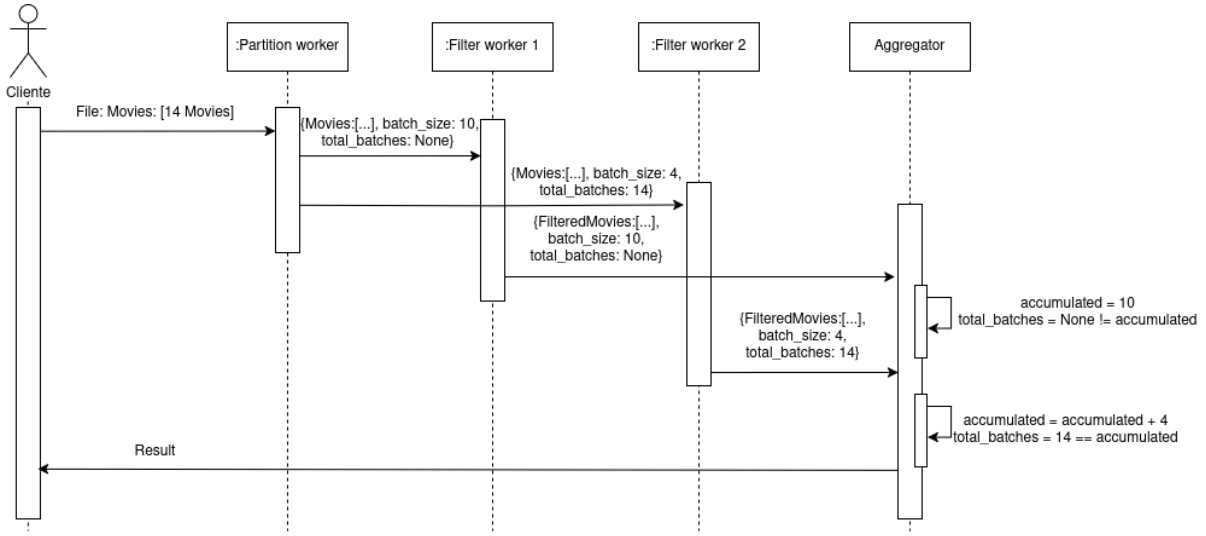
Diagrama de secuencia

Luego de que el cliente dispare la request, el cliente será el encargado de enviarla a través del middleware al filtro principal el cual realizará una limpieza del dataset y devolverá únicamente las columnas que se utilizaran. Luego este irá mandando a los filtros necesarios. En este caso primero pasa por 20th century de producción argentina, para luego continuar su flujo en el de producción española. Una vez aquí Se continua en el aggregator, este recibe datos hasta que no haya más en la cola. Todos estos mensajes se realizan a través del middleware. Finalmente el aggregator responde al cliente.

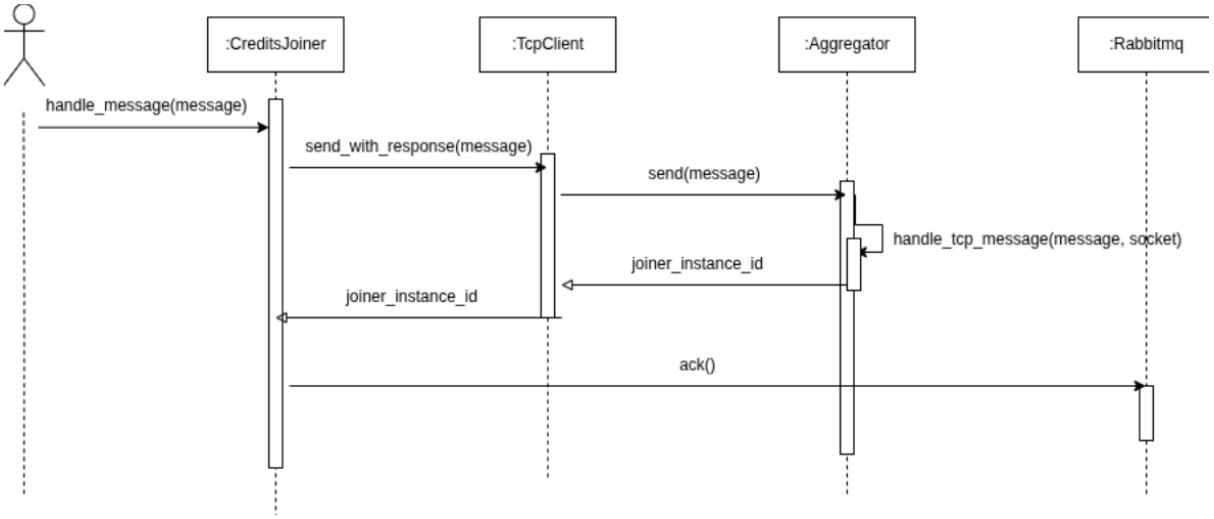
Diagrama representativo de la consulta 1: Películas y países de los años 2000 con producción argentina y española.



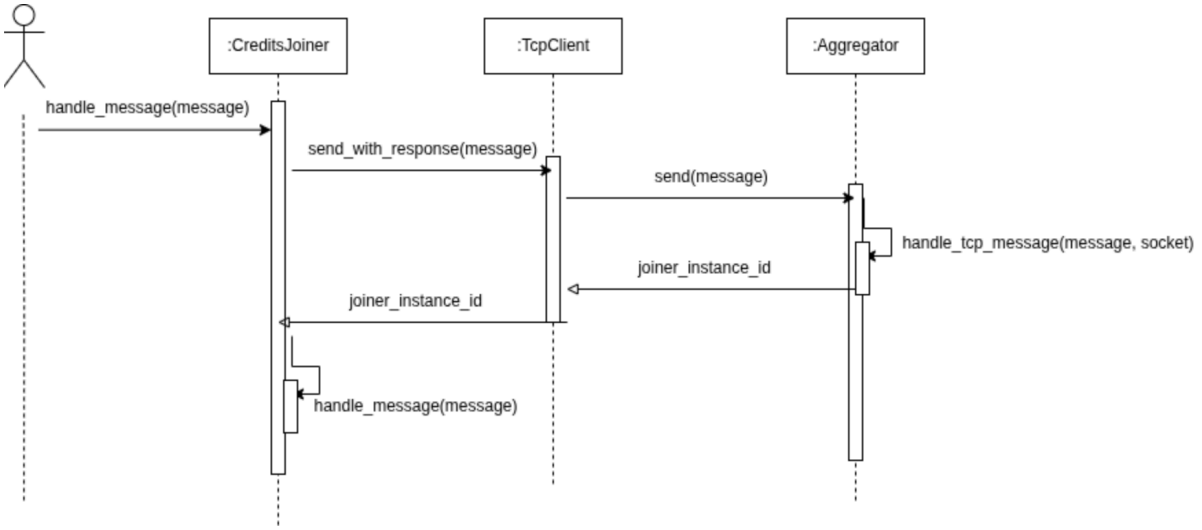
El siguiente diagrama de secuencia ilustra el comportamiento del sistema ante un EOF del archivo.



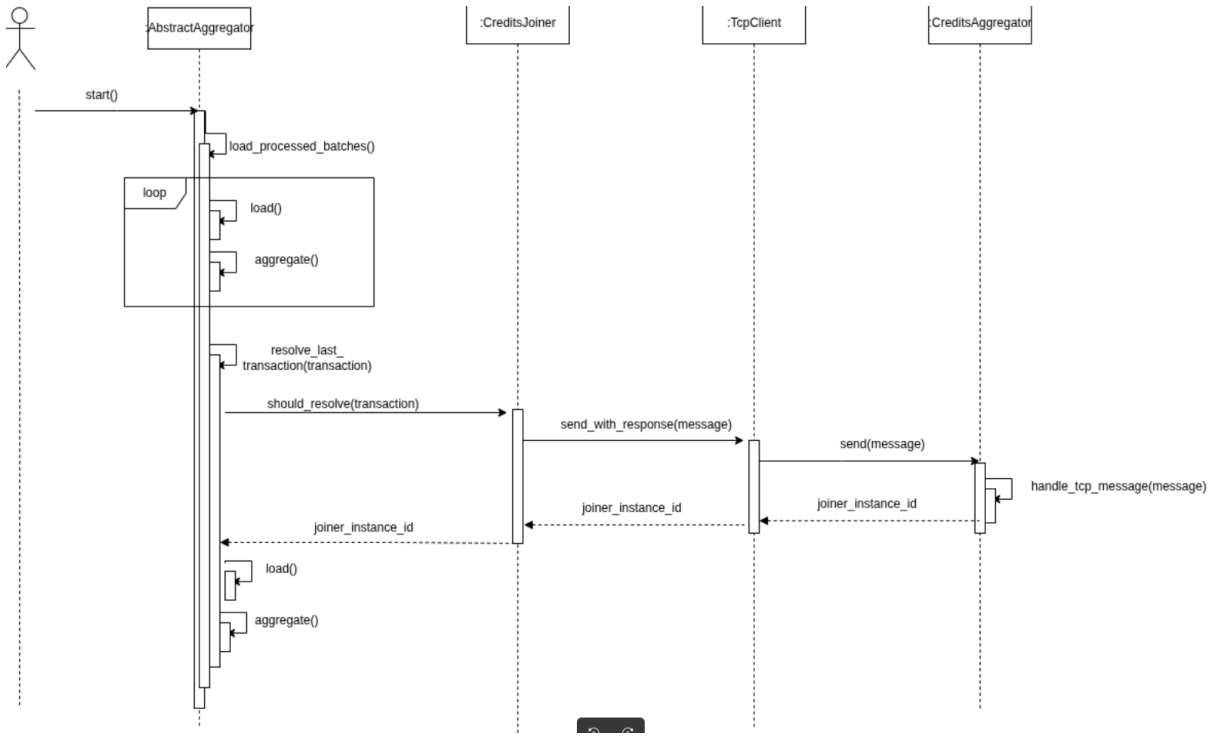
En el siguiente diagrama de secuencia, se ilustra una de las comunicaciones realizadas mediante tcp. En esta el joiner recibe un archivo de credits/ratings y quiere consultar si este ya fue procesado o no. Es el aggregator quien le retorna esta respuesta indicando un joiner_id de quien fue quien realizó el procesamiento. Dado que se retorno un joiner_id no propio, se envía un ack a rabbitmq para avisar que ya fue procesado y no vuelva a enviarlo.



A diferencia del caso anterior, aqui no mandaremos un ack a rabbit ya que no llego a procesarse. En esta situacion continuamos aquella ejecucion.



El siguiente diagrama muestra la situación detallada en la sección TCP::Inicio de AbstractAggregator. Aquí se puede apreciar un loop para todas las transacciones a excepcion de la última quien se trata de manera particular, en base a si tiene asociado su END_TRANSACTION o no.

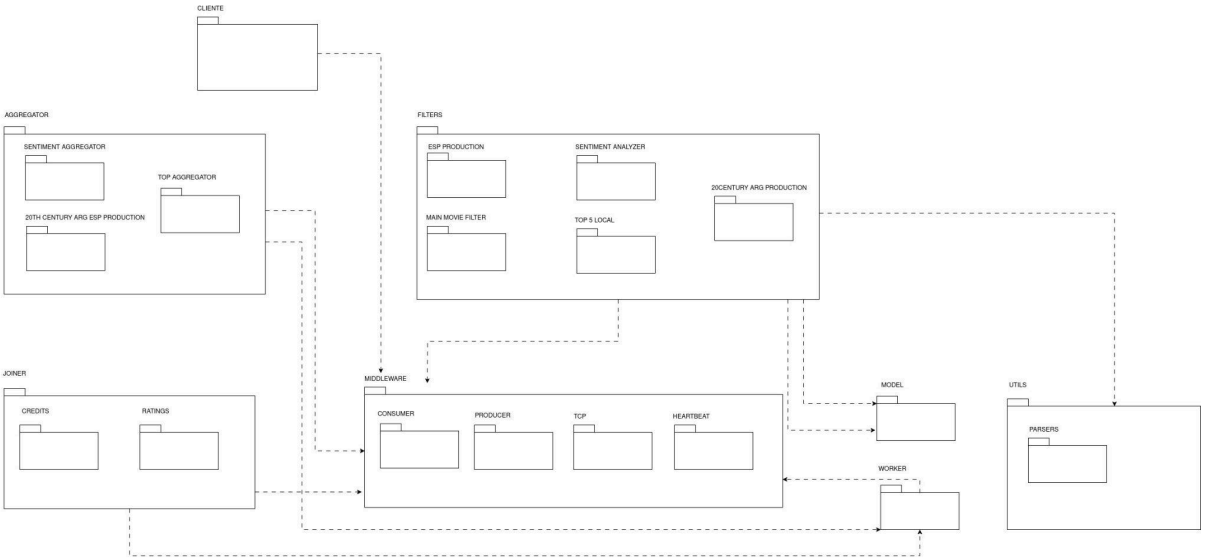


Vista de desarrollo

Diagrama de paquetes

Como paquetes principales definimos:

- El respectivo al middleware con la implementación tanto del consumer como del producer
- El asociado al cliente
- Aggregator el cual dentro tendrá al particular de cada uno
- Filters para el que rige la misma regla
- Joiners el cual almacena tanto el de credits como el de ratings
- Model para definir entidades
- Worker para utilizar herencia dentro de cada uno de ellos
- Utils



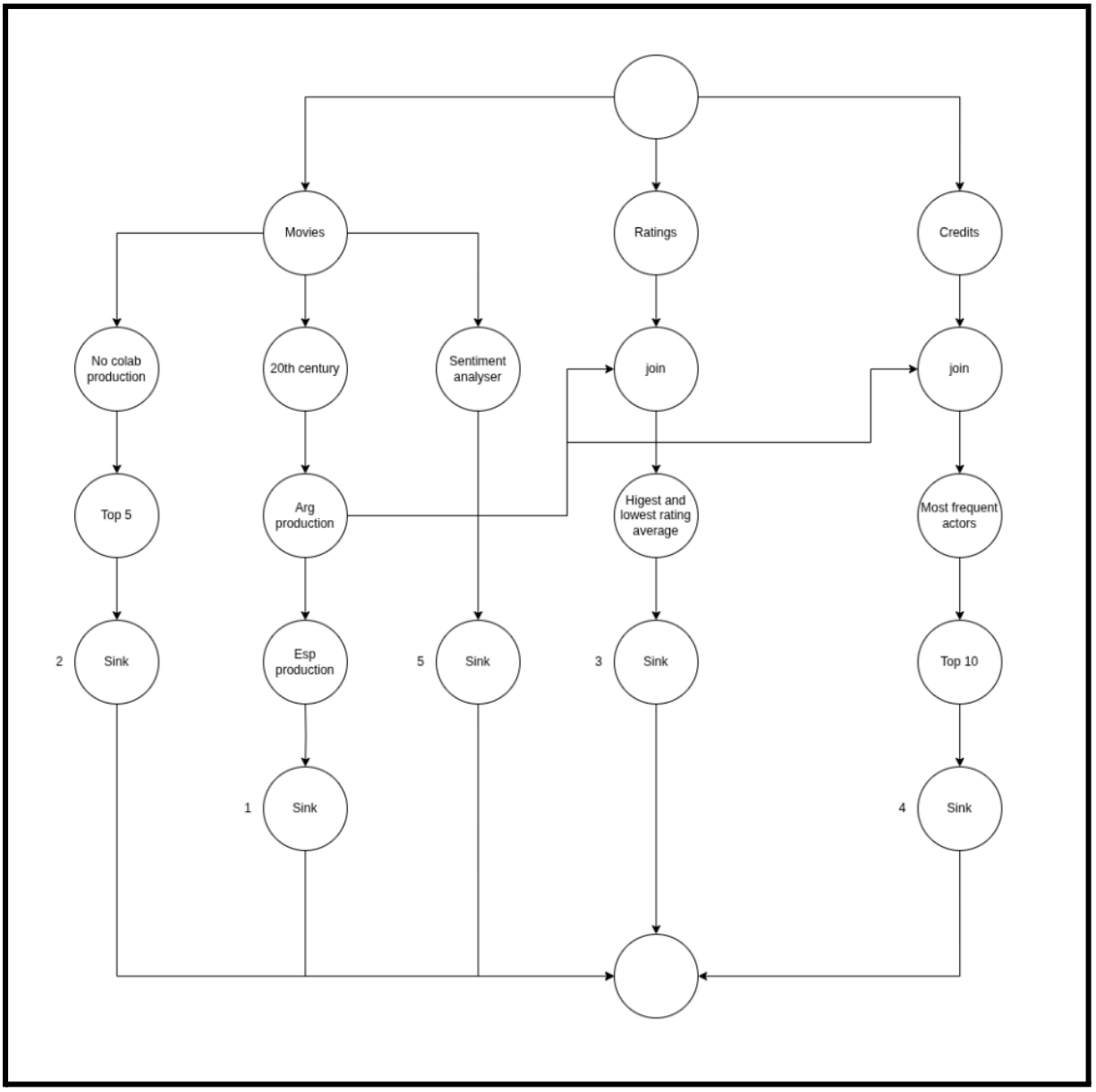
DAG

Las primeras dos consultas se pueden resolver a partir del archivo de movies únicamente.

Para la tercera consulta, se realiza un primer filtro y el resultado de este se joinea con el archivo de ratings, para luego continuar aplicando los filtros restantes.

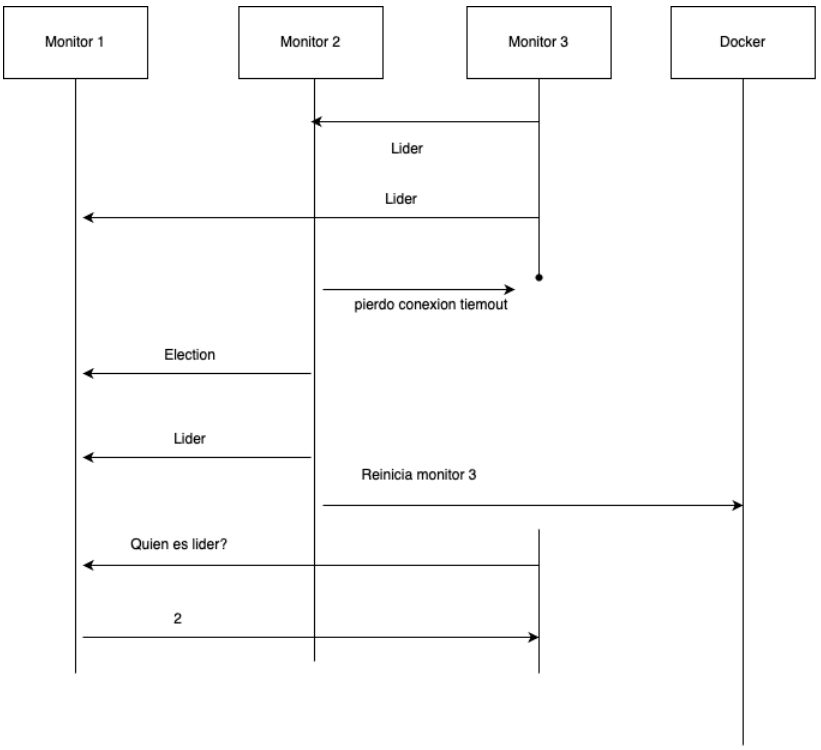
La query 4 utiliza al mismo filtro que esta última (producción argentina a partir de los años 2000) y luego hace un join pero con el archivo de credits. Nuevamente luego de esto se realizan nuevos filtros hasta llegar al sink.

Finalmente, la última consulta luego de aplicar el filtro realiza el análisis de sentimientos.



Otros

Eleccion de lider


Monitor 1
Monitor 2
Monitor 3
Docker
Lider
Lider
pierdo conexion tiemout
Election
Lider
Reinicia monitor 3
Quien es lider?
2